Fine-Tuning a Small Language Model to Classify Messy Engineering Bill-of-Materials Text into US Harmonized Tariff Codes

Navya Nitin Maheshwari
Department of Computer Science and Engineering
Shiv Nadar University Chennai, India
Roll Number: 24110301
Email: navya24110301@snuchennai.edu.in

***Abstract*—Every item that is imported into the United States needs to have a 10-digit tariff code. This is also known as a Harmonized Tariff Schedule code of the United States (HTSUS), and helps decide the duty paid on the product. Recently, researchers have fine-tuned LLM's on clean product descriptions, such that the right HTS code is returned (generated) when required. These product descriptions range over various classes such as food, chemicals, manufacturing, animal descriptions, and so on. In manufacturing, product or part descriptions often tend to be messy. This is an area that has not been explored deeply when it comes to HTSUS classification. This project was done with the aim of experimenting whether a small model can classify such messy text, for example, an abbreviated bill-of-materials line written by an engineer. I built a dataset of over 4025 product descriptions from across 111 open-source hardware projects. Using Mouser and Digi-key distributor APIs, I labelled 2,891 distinct descriptions with 116 distinct US tariff codes through their part numbers. The train, test, and validation test sets have been split according to each project, instead of across rows. This is so that the test set contains writing styles that have never been seen during training. I have fine-tuned Qwen2.5-3B-Instruct with QLoRA, by running it in colab on 1 T4 GPU (free). I compared this against nearest-neighbour baseline and the Qwen zero-shot (not fine-tuned) model. The test set eventually has 237 descriptions (rows). On this, the model reached exact 10-digit (HTSUS code) accuracy of 64.6%, and 4-digit (heading) accuracy of 87.8%, against the comparatively smaller baselines. Contributions are messy-text dataset, an evaluation protocol that ensures there's no leak between train and test datasets, and lastly, a low-cost fine-tuning recipe.**
Code repository:
https://github.com/nzoopxw/hts-messy-bom-qwen

***Index Terms*—tariff classification, Harmonized Tariff Schedule, large language models, QLoRA, noisy text, bill of materials**

I. Introduction

Any goods that cross the US border must be classified under the Harmonised Tariff Schedule of the United States (HTSUS). Each product has a 10-digit tariff code which determines duty, and the descriptions of these products are hierarchical. 2 indicates the chapter, 4 the heading, 6 the international subheading, 8 the US tariff line that sets duty, and 10 is the US statistical suffix. This hierarchy spans over 17,000 pages, and classification is still largely done by hand. A misclassification could lead to overpaid duty, penalties, and Delays in shipment. Studies done recently show that LLMs fine-tuned on labelled examples of these descriptions and their codes can learn this task of classification. [1], [2] Research that has been published currently shows that models have been trained and evaluated on clean descriptions. That is, clearly worded phrases and the HTSUS codes corresponding to them. A class that has not been as deeply explored is manufacturing. In a manufacturing company, the input is a bill-of-materials (BOM) line written by an engineer, and is meant for other engineers. For example, `100n Bypass 16V/20% C_0402` or `600R L_0402`. Lines like these are short, abbreviated, and the styles vary across authors. Whether the models trained on different classes and clean descriptions cope with this type of input – that is something that has not been measured yet. This project is **research-paper-based**. The source paper is ATLAS [1], presented at a NeurIPS 2025 workshop, which fine-tuned LLaMA-3.3-70B on 18,731 US Customs rulings. While US customs rulings have labels verified by official sources, my dataset has Harmonized Tariff Schedule Codes that have been confirmed with Mouser and Digi-key distributor APIs with a match of 98.4% on all 10-digits, and 100% on 4-digits. The goal here is to not reproduce ATLAS, rather to extend it's approach in 3 main direction that authors leave open, and that has been recommended by reviewers: messy, real-world input text, using a much smaller model, and a clean train-test-validation split that is completely separate from one another i.e., has no overlap. My hypothesis is that a 3B-parameter model fine tuned with quantized low-rank adaptation (QLoRA) [5] on messy BOM text outperforms both neighbour (lexical) retrieval and the zero-shot model of Qwen.

The contributions are:

1) A dataset of 2,981 real engineer-written BOM descriptions from open-source hardware projects, labelled with 116 distinct US 10-digit codes through two distributor databases.
2) A leakage-free evaluation protocol: splitting by project, removing shared part numbers, and scoring every level of the code hierarchy.
3) A fine-tuning recipe for Qwen2.5-3B [7] that runs on

one free 16 GB GPU, compared against two baselines. Section II reviews related work, Section III describes the dataset and method, Section IV reports results, and Section V concludes.

II. Related Works

ATLAS [1] built a dataset from the US Customs Rulings Online Search System (CROSS). This consisted of 18,731 rulings that covered 2,992 distinct 10-digit codes. 200 rulings, each for validation and test. Each ruling was condensed by GPT-4o-mini into 3 parts. First, a short product description, second a reasoning trace and lastly, the code. The model thus reads clean, complete descriptions, such as "Silicon wafers, partially processed, used for semiconductor fabrication." Full fine-tuning of LLaMA-3.3-70B on 16 A100-80GB GPUs reached 40.0% 10-digit and 57.5% 6-digit accuracy, ahead of GPT-5-Thinking (25.0% and 55.5%) and Gemini-2.5-Pro-Thinking (13.5% and 31.0%). The untuned LLaMA-3.3-70B scored only 2.1% and 20.7%, showing that the gain came from fine-tuning. Its limitations are that every input is a clean ruling description, only one 70B model was studied (the authors list distilling ATLAS into smaller 8B–3B variants as future work), and the 200 test rulings are drawn from the same pool as training, stratified by chapter.

Commercial tools. Judy [2] evaluated Zonos, Tarifflo, Avalara and the World Customs Organization's BACUDA on 103 items from CBP rulings. Tarifflo scored highest at 10 digits (89%) and Zonos 44%. BACUDA reached 13% at 6 digits. The benchmark is small, uses ruling text, and its author also built the top-ranked tool.

HSCodeComp [3] is a benchmark of e-commerce product listings for deep-search agents that must apply the hierarchical HS rules. It shows that current agents remain far below human experts. Its inputs however, are shopper-facing listings and not engineering shorthand descriptions.

HSGraphAgent [4] guides an LLM step-by-step, through a knowledge graph of the HS hierarchy. This enforces containment and exclusion rules so that the model stays on legally valid paths. It does not use fine-tuning.It runs 4 backbones zero-shot and following the graph, the the locally deployed Qwen2.5-32B reaches 85.9% at 4 digits and 77.8% at 6 digits on international HS codes for product descriptions. My earlier prototype followed a related design, in which the HTS hierarchy was stored as a Neo4j graph. GPT-5 then converted the engineering shorthand descriptions into search terms. GPT-5 converted shorthand into search terms, and candidates were retrieved from the graph. Its top three candidates agreed with GPT-5's own answer at 4 digits for 60% of inputs. Without ground-truth labels this was a measure of agreement, not accuracy. The present work instead builds labelled data and fine-tunes a Qwen model ten times smaller than HSGraphAgent's, and that too for full US 10-digit codes.

Parameter-efficient fine-tuning. LoRA [6] freezes a pre-trained model and learns low-rank updates to its weight matrices. In turn, this reduces the number of trained parameters, with little loss in quality. QLoRA [5] adds 4-bit NormalFloat quantisation of the frozen weights, double quantisation and paged optimisers, showed that this matches 16-bit fine-tuning while fitting large models on a single GPU. These methods make it practical to adapt a model such as Qwen2.5 [7] to a narrow task on free hardware. This is something that ATLAS is looking forward to addressing in the future.

TABLE I
Input text used by prior work and by this project.

Work	Input text	Labels	Model
ATLAS [1]	CBP rulings	US 10-dig.	LLaMA-70B, full FT
Judy [2]	CBP rulings	US 10-dig.	Commercial tools
HSCodeComp [3]	Shop listings	HS	Search agents
HSGraphAgent [4]	Descriptions	HS 4/6-dig.	LLM + graph
My Work	**Engineer BOM**	**US 10-dig.**	**Qwen-3B, QLoRA**

TABLE II
Levels of a US HTS code, illustrated with a multilayer ceramic capacitor.

Digits	Level	Example	Meaning
2	Chapter	85	Electrical machinery and equipment
4	Heading	8532	Electrical capacitors
6	HS subheading	8532.24	Ceramic dielectric, multilayer
8	US tariff line	8532.24.00	Sets the duty rate
10	Statistical suffix	8532.24.0020	US statistical reporting

Table I summarises the difference: all prior inputs are written to be readable by a non-expert, while ours are not.

III. Proposed Methodology

A. Problem Formulation and Dataset

Task. The input is one BOM description string x, written by an engineer. The output is its 10-digit US HTS code y (e.g. `8548.00.0000`). The task is a single-label classification framed as text generation. HTS codes are hierarchical (Table II), so a prediction can be partly right. The starting digits fix the type of article, and the digits after that refine it.Classification is generative here, so the model can, in principle, return a code that it has never seen in training before.

Source. I have cloned 169 repositories (open-source, hardware repos) from GitHub, such as the Kitspace project registry [8] along with well-known boards such as HackRF, the Antmicro Jetson baseboards, and the NASA JPL Open Source Rover. A parser reads KiCad schematics and BOM spreadsheets, and extracts each part's value, description, specification fields, package and manufacturer part number (MPN). This yielded 4,025 unique rows (project, part, description) from 111 projects,covering 3,125 distinct MPNs.

Annotation. Each MPN was searched for in the Mouser Search API and the Digi-Key Product Information V4 API, both of which publish a US HTS code for most parts. A label is *gold* when both distributors give the same 10-digit code, *single* when only one does, and *conflict* when they disagree. 2,301 of the 3,125 parts received a 10-digit code, covering 2,981 of the 4,025 rows and 116 distinct codes, within the free daily API quotes. The remaining parts had discontinued or

TABLE III
REPRESENTATIVE INPUTS FROM THE DATASET.

Input text (as written)	Item
`6n8`	6.8 nF capacitor
`600R L_0402`	Ferrite bead
`100n Bypass 16V/20% C_0402`	Ceramic capacitor
`3V3REG LDO CMOS HICURR U-DFN3030-8`	3.3 V linear regulator
`R_200R_0402 1% R_0402`	Chip resistor

TABLE IV
DATASET SPLITS. SPLITS ARE DISJOINT BY PROJECT; AUGMENTED ROWS APPEAR IN TRAINING ONLY.

Split	Projects	Real rows	Augmented	Codes	Headings
Train	50	1,176	919	108	29
Validation	15	164	0	38	9
Test	26	237	0	53	16

mistyped numbers, or were not queried within the quota. Most labels are therefore single-source. A 190-part pilot labeled by both distributors, which agreed on 98.4% of parts at 10-digits. And all parts at 4 digits. The manufacturer part number is only used to find the label. The model's input is build completely fro m the text that the engineer wrote.

Messiness. The median description is 20 characters. Only 55% name the item in words. 25% contain KiCad footprint codes. 39% lead with a component value (Table III).

Split and leakage precautions. Rows are split by each project. As each project has its own writing style owing to the difference in writing styles of different engineers. 50-26-15 is the train-test-validation set split. This ensures that the test set can check whether a model can read styles it's never seen before. A random split would leak styles and end up inflating the score. Furthermore, any validation or test row whose MPN also occurs in training has been removed. Codes have also been capped at 80 rows in training and 20 rows in evaluation so that parts such as resistors and capacitors do not dominate. De-duplication was also done. Table IV gives the final splits over 91 projects. Out of 111 projects, 91 are the number of projects that, after labeling and cleaning still had 1 row left.

Six test rows (2.5%) carry a code never seen in training, and one of them have a heading never seen in training, so a small part of the test set requires generalising beyond the training labels. Only 2 test rows were copied from a catalogue, so 235 of 237 test rows are genuinely engineer-written.

B. Processing Pipeline and Approach

Fig. 1 shows the pipeline. Harvesting and labelling produce the dataset; QLoRA fine-tuning and the two baselines are then scored on the same held-out projects.

Input formatting. Each example is a chat with a fixed system instruction ("reply with only its 10-digit US HTS code in the form XXXX.XX.XXXX"), the BOM text as the user turn, and the code as the assistant turn. The loss is computed

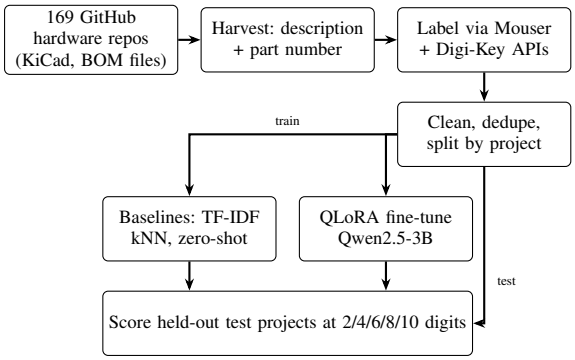


Fig. 1. Dataset construction and evaluation pipeline. Part numbers are used only for labelling and never reach the model.

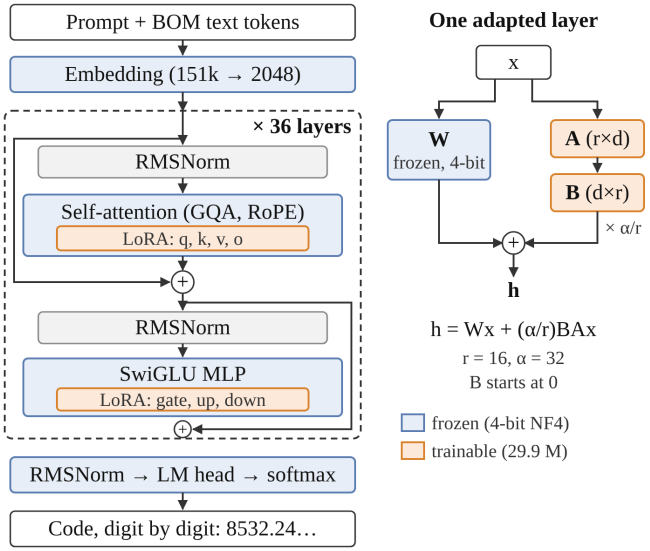


Fig. 2. Qwen2.5-3B with QLoRA adapters. Each of the 36 decoder layers applies RMSNorm, grouped-query self-attention and a SwiGLU MLP with residual connections. The pretrained weights are frozen in 4-bit NF4, and low-rank adapters A and B (rank 16) are trained on the q, k, v, o, gate, up and down projections (29.9 M parameters).

only on the assistant tokens:

$$\mathcal{L} = -\sum_{t \in \mathcal{A}} \log p_\theta(w_t \mid w_{<t}), \tag{1}$$

where $\mathcal{A}$ indexes the answer tokens.

QLoRA. The base weights are frozen in 4-bit NormalFloat (NF4) with double quantization [5]. Low-rank adapters [6] are added to every attention and MLP projection (q, k, v, o, gate, up, down), For a frozen matrix W, the adapted layer computes

$$h = Wx + \frac{\alpha}{r} BAx, \quad A \in \mathbb{R}^{r \times d}, \; B \in \mathbb{R}^{d \times r}, \tag{2}$$

and only A and B are trained, which lets a 3B model train on one 16 GB GPU.

Augmentation (training only). Each training row receives one synthetic enterprise-resource-planning (ERP) style variant -> common words abbreviated (e.g. capacitor → CAP),

TABLE V
PIPELINE STAGES, CODE AND MEASURED COST.

Stage	Script / notebook	Cost
Harvest 169 repos	`harvest_boms.py`	≈30 min
Mouser labelling	`label_parts.py`	221 calls, ≈10 min
Digi-Key labelling	`label_parts.py`	900 calls
Build splits	`build_dataset.py`	seconds
QLoRA training	notebook	24.1 min, T4
Ablation training	notebook	12.1 min, T4

uppercased, comma-separated and truncated to 30 characters. Validation and test sets contain only real text.

Inference. Greedy decoding, with at most 16 new tokens. The first code-shaped string in the output is taken as the prediction.

Baselines. (1) *TF-IDF nearest neighbour*: character 2–4-gram TF-IDF vectors with cosine similarity, which predicts the code of the closest training description. (2) *Zero-shot*: the same Qwen2.5-3B-Instruct model and prompt, just without fine-tuning. To isolate training effect.

Changes relative to ATLAS. ATLAS performs full fine-tuning of a 70B model on ruling text and also generates a reasoning trace. We use parameter-efficient QLoRA on a 3B model, train on messy BOM text with ERP-style augmentation, output the code only, and split by project instead of at random. The QLoRA method itself is not new; the contribution lies in the data, the protocol and the setting.

C. Implementation Details

Python 3.9 (data pipeline) and Python 3.13 on Google Colab (training) were used with PyTorch, Hugging Face Transformers (≥4.44), PEFT (≥0.12), bitsandbytes (≥0.43), scikit-learn and pandas. Training ran on one NVIDIA T4 (16 GB). Hyperparameters: LoRA rank 16, $\alpha = 32$, dropout 0.05, paged 8-bit AdamW, learning rate 2×10^{-4} with cosine decay and 5% warm-up, batch size 16, epochs = 3, fp16 compute, seed 42. The adapters add 29.9 M trainable parameters (0.96% of the 3.1 B total), and training took 24.1 minutes. The repository contains the harvesting, labelling, dataset-building and notebook code with a README; Table V lists each stage, its script and its cost. API keys are read from a local `.env` file that is excluded from version control. Also, every API response is cached so that each stage can be re-run without repeating calls. Zero-shot inference over the 237 test rows took 18 s (about 76 ms per row in batches of 32).

IV. RESULTS AND DISCUSSION

A. Experimental Setup and Evaluation

HTS codes are hierarchical: 2 digits give the chapter, 4 the heading, 6 the international subheading, 8 the US tariff line and 10 the statistical suffix. The accuracy at depth k, the percentage of test rows whose first k predicted digits equalled to the label, for $k \in \{2, 4, 6, 8, 10\}$. This shows where errors occur. $k = 6$ and $k = 10$ are directly comparable with ATLAS. Two validity measures are added. One, being the share of outputs containing a 10-digit code (well-formed) and second, the share whose code occurs among the dataset's labels, a proxy for real codes. As only 2 of the 237 test rows were pasted from a catalogue, the results on all the rows are effectively results on the engineer-written text, and no separate copied-row score is reported. The validation set is used only as a post-training sanity check, and all reported numbers are final held-out test results.

TABLE VI
TEST ACCURACY (%) AT EACH CODE DEPTH ON 237 ROWS FROM 26 HELD-OUT PROJECTS. AVG: MEAN NUMBER OF LEADING DIGITS CORRECT (0–10), ATLAS'S THIRD METRIC. WF: SHARE OF OUTPUTS THAT ARE A WELL-FORMED 10-DIGIT CODE. HIGHER IS BETTER.

Method	@2	@4	@6	@8	@10	Avg	WF
TF-IDF kNN	97.0	81.4	69.6	69.6	58.6	7.68	100
Qwen2.5-3B zero-shot	11.8	1.7	0.0	0.0	0.0	0.35	86.5
Qwen2.5-3B QLoRA, run 1	98.3	86.5	76.8	75.5	63.3	–	100
Qwen2.5-3B QLoRA, run 2	**98.3**	**87.8**	**76.8**	**75.5**	**64.6**	**8.17**	100

B. Quantitative and Qualitative Results

Table VI compares the methods on the 237 rows of the 26 held-out test projects. The fine-tuned model was trained twice with identical settings; run 2 also saved per-row predictions, so it is used for all analysis below. The fine-tuned model is the most accurate at every depth.In run 2, 64.6% of its predictions matched all 10 digits and 87.8% matched the 4-digit heading. This was against 58.6% and 81.4% for the nearest-neighbour baseline. The two runs differed by at most 1.3 points, which indicates the run-to-run variation.

Statistical significance. Since all methods were scored on the same rows, I used an exact McNemar test on them - where exactly on eout of two methods is correct. Against the baseline for the nearest-neighbour, the fine-tuned model was right on the 27 rows where the baseline was wrong. It was also wrong on the 12 rows where the baseline was right at 4 digits ($p = 0.024$), and 32 versus 15 at 6 digits ($p = 0.019$). The split was 38 vs 24 at 10 digits($p = 0.098$). Suggestive but still not significant at 5% level.

Training behaviour. Fine-tuning took 393 optimisation steps (3 epochs) and 24.1 minutes on one T4. The training loss fell from 1.21 to 0.05 (Fig. 3), and it averaged 0.196 over the run. In run 1, on the held-out validation projects, the model scored 63.4% at 10 digits and 90.2% at 4 digits, which was close to its test scores. Hence it did not overfit to the writing styles and patterns of the training projects.

Accuracy by part type. Table VII breaks down 10-digit accuracy by the heading of the true code, and fine-tuning helped most with capacitors (+20.0 points), integrated circuits (+10.0), connectors and switches (+10.3) and resistors (+11.1). It was worse than retrieval on inductors and slightly worse on discrete semiconductors.

Sample predictions. Table VIII shows representative test rows. The fine-tuned model read units and context that the character matching missed -> it recognised `390nH` as an

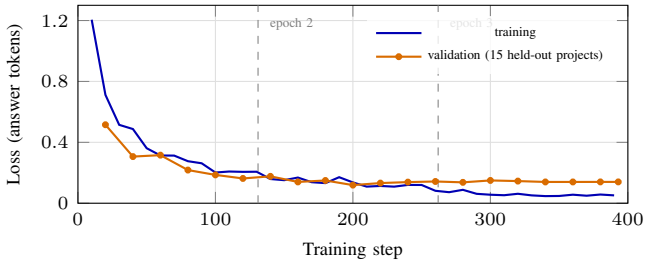


Fig. 3. Training and validation loss for Qwen2.5-3B-Instruct with QLoRA, from a diagnostic run with the same settings as the main runs. Loss is computed on the answer tokens only. Dashed lines mark epoch boundaries.

TABLE VII
EXACT 10-DIGIT ACCURACY (%) BY TRUE HEADING, FOR HEADINGS WITH AT LEAST 6 TEST ROWS (RUN 2).

Heading	Part type	Rows	kNN	QLoRA
8541	Diodes, transistors, LEDs, crystals	54	**66.7**	63.0
8542	Integrated circuits	50	38.0	**48.0**
8536	Connectors, switches, sockets	39	56.4	**66.7**
8532	Capacitors	35	68.6	**88.6**
8533	Resistors, potentiometers	27	74.1	**85.2**
8504	Inductors	11	**81.8**	63.6
8548	Ferrite beads	6	83.3	**100.0**

inductor and `4.7uf tantalum 3528` as a tantalum capacitor. These were both assigned to the wrong heading by the baseline.

C. Discussion and Limitations

Why the zero-shot model fails. Without fine tuning, Qwen2.5-3B produced a string for 86.5% of rows, that was code-shaped. However, none of those codes occurred anywhere in the dataset's label set. Infact, only 1.7% has the right heading. often, its outputs were malformed eg. `8543.901000`, `2006.9090.00`. A general 3B model does not have tariff knowledge or the ability to decode shorthand like `600R L_0402`. With it's own untuned base model, ATLAS observed the same. It had 2.1% exact accuracy [1]. So, this failure is not specific to small models.

Why fine-tuning beats retrieval. fine-tuning adds the most at the 4- and 6- digit levels, where a model should recognise *what kind* of part the string is describing rather than its characters. Because many parts recur across projects in similar wording, the nearest neighbour baseline is strong. It already gets the chapter right for 97% of rows. The N-grams baseline treats `10k R_0402` and `10k C_0402` as near identical, whereas on the other hand, the fine-tuned model has learned that the footprint prefix and the unit (`nH`, `uF`, `R`) decide the heading. on capacitors, it had its largest gain on separating tantalum (8532.21), aluminum (8532.22) and ceramic (8532.24).

Where the remaining errors are. Accuracy barely changes from 6 to 8 digits (76.8% vs. 75.5%) but falls by 10.9 points from 8 to 10 digits. Of the 26 rows that are right at 8 digits but wrong at 10, the commonest confusion is between

TABLE VIII
SAMPLE TEST PREDICTIONS (RUN 2). ✓/×: CORRECT/INCORRECT AT 10 DIGITS.

Input text	True code	kNN	QLoRA
`390nH`	8504.50.8000	8541.41.0000 ×	8504.50.8000 ✓
`4.7uf tantalum 3528`	8532.21.0050	8532.24.0020 ×	8532.21.0050 ✓
`FerriteBead L_0805_2012Metric`	8548.00.0000	8533.21.0030 ×	8548.00.0000 ✓
`18650 Battery holder ...`	8536.90.8585	8547.20.0000 ×	8536.90.8585 ✓
`Fixed 6V 3.3V 600mA SOT-25 LDO`	8542.39.0090	8542.39.0090 ✓	8542.39.0070 ×
`SMAJ24A 24V TVS 400W SMA`	8541.10.0080	8541.10.0080 ✓	8536.10.0050 ×
`784383240047 L_WE-MAIA-2512`	8504.50.8000	8504.50.8000 ✓	8536.90.4000 ×
`0805`	8533.21.0030	8533.21.0030 ✓	8532.24.0020 ×

TABLE IX
THE 29 TEST ROWS THE FINE-TUNED MODEL GOT WRONG AT THE 4-DIGIT HEADING LEVEL (RUN 2), GROUPED BY CAUSE.

Cause	Rows	Example input
Rare type (modules, boards, RF)	16	`RFM95 HopeRF_RFM9X`
Part number only	7	`USB2514B`
Package code only	4	`0805`
Protection-device confusion	2	`SMAJ24A 24V TVS`

the statistical suffixes 8542.39.0070 and 8542.39.0090 for integrated circuits, which depend on the exact circuit function rather than anything written in a BOM line. These errors matter little in practice, because duty is set at the 8-digit tariff line. The 29 heading-level errors fall into four groups: (1) text with no identifying content, such as a bare package code (`0805`, `0201`, `SOT-23-5`), where the same string even carries different true codes in different projects; (2) lines that are only a part number (`USB2514B`, `784383240047`), which carry no description, so the model can only guess from the footprint; (3) rare product types such as radio modules, development boards, antennas and microphones (headings 8517, 8471, 8518), which have few or no training examples; and (4) protection devices, where TVS diodes (8541.10) were confused with fuses (8536.10). Inductors are a weakness because several test inductors are written only as a part number plus footprint. Table IX counts the four groups.

Ablation: ERP-style augmentation. To measure the component we added to the training data, the model was retrained with identical settings on the 1,176 real training rows only (222 steps, 12.1 minutes). Table X compares it with run 2 on the same test set.

Augmentation raised exact 10-digit accuracy by 5.5 points: it fixed 24 rows and broke 11 ($p = 0.041$, McNemar). Its 1.2-point loss at 4 digits (3 rows gained, 6 lost) is not significant ($p = 0.51$). The extra spelling variants appear to give the model more examples per code, helping it separate codes that differ only in their last digits, while truncated variants occasionally remove the word that identifies the heading. Its tantalum-capacitor predictions illustrate this: the model without augmentation labelled `33uF tantalum 7343` and `10uF 35V Tantalum 7343` as aluminium capacitors (8532.22), while the augmented model got both right.

Comparison with ATLAS. Our 64.6% (10 digits), 76.8% (6 digits) and 8.17 average correct digits are higher than

TABLE X
ABLATION OF ERP-STYLE AUGMENTATION (TEST ACCURACY, %). BOTH RUNS USE QWEN2.5-3B, QLORA AND IDENTICAL HYPERPARAMETERS.

Training data	@2	@4	@6	@8	@10
Real + augmented (2,095 rows)	98.3	87.8	**76.8**	**75.5**	**64.6**
Real only (1,176 rows)	98.3	**89.0**	76.4	74.3	59.1
Difference	0.0	−1.2	+0.4	+1.2	+5.5

ATLAS's 40.0%, 57.5% and 6.30, but the two are not directly comparable: ATLAS covers 2,992 codes across every chapter, while our electronics label space has 116 codes concentrated in about 30 headings. What the comparison does show is that a model about 23 times smaller, trained in 24 minutes on a free GPU, can handle the messier input that ATLAS does not test.

Limitations. Several limitations bound these results. The dataset covers electronic components only; packaging, fabricated metal parts and plastic covers common in a factory item master are absent. Labels come from distributors rather than customs rulings, and although the two sources agreed on 98.4% of pilot parts, shared errors would go undetected. Electronics concentrate in a few headings (8532 capacitors, 8533 resistors, 8541–8542 semiconductors, 8536 connectors), so the label space is far smaller than ATLAS's 2,992 codes and absolute accuracies are not directly comparable in difficulty. Finally, only one model size was trained, twice, because of the compute budget.

V. CONCLUSION AND FUTURE WORK

Engineers write short, abbreviated part descriptions. The aim of this project was to test whether a small, ope model could assign US tariff codes to these. A dataset of such descriptions was built, from open-source hardward projects on GitHub. It was then labelled through distrubitor databases, and finally evaluated on projects never seen in training. Qwen2.5-3B fine-tuned with QLoRA on one free GPU reached 64.6% exact 10-digit and 87.8% 4-digit accuracy, ahead of a character n-gram retrieval baseline (58.6% and 81.4%). The same model without fine-tuning was almost never right. Fine tuning therefore supplies both the tariff knowledge and the ability to read engineering shorthand. Its advantage over retrieval is the largest at the heading and the subheading levels of the hierarchy. An ablation showed that ERP-style augmentation improved hhe exact vode accuracy by 5.5 points.

Two limitations matter most: 1) electronics-only coverage, and 2) distributor-derived labels. Three next steps follow right after this - (1) Cover more than electronics, adding packaging, metal and plastic parts so the model handles a whole factory parts list. (2) Never output a code that does not exist, by checking each prediction against the real HTS schedule stored in the earlier Neo4j graph, combining the graph and fine-tuning approaches. (3) Try a larger model, training Qwen2.5-7B the same way over several seeds to measure what extra size adds.

REFERENCES

[1] P. Yuvraj and S. Devarakonda, "ATLAS: Benchmarking and adapting large language models for global trade via harmonized tariff code classification," in *NeurIPS 2025 Workshop on Knowledge Graphs and Agentic Systems Interplay (NORA)*, 2025. [Online]. Available: https://arxiv.org/abs/2509.18400

[2] B. Judy, "Benchmarking harmonized tariff schedule classification models," *arXiv preprint arXiv:2412.14179*, 2024.

[3] Y. Yang, T. Lan, Q. Jia, L. Zhu, H. Jiang, H. Zhu, L. Wang, W. Luo, and K. Zhang, "HSCodeComp: A realistic and expert-level benchmark for deep search agents in hierarchical rule application," *arXiv preprint arXiv:2510.19631*, 2025.

[4] Q. Xia, Z. Zhang, A. Wang, W. Wang, X. Wang, and J. Li, "HSGraphAgent: Knowledge-graph-guided large language models for harmonized system code classification," in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2026, pp. 44761–44773, doi: 10.18653/v1/2026.acl-long.2072.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[6] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[7] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[8] Kitspace, "Kitspace: A place to share electronics projects," 2024. [Online]. Available: https://github.com/kitspace/kitspace